

Competitive Programming

Section 30 Editorial: GCD, LCM, and Diophantine Equations

Competitive Programming Course

2026-08-14

Table of contents

1	Editorial map	4
2	Shared tool: extended Euclid	5
2.1	Shared C++ helper	5
2.2	Shared Python helper	5
3	A. GCD LCM Queries	6
3.1	Restatement and traps	6
3.2	Derivation and pseudocode	6
3.3	Worked trace	6
3.4	Correctness proof	7
3.5	Complexity	7
3.6	C++ and Python notes	7
3.7	Wrong answers and targeted tests	7
4	B. General Inverse	8
4.1	Restatement and traps	8
4.2	Derivation and pseudocode	8
4.3	Worked traces	8
4.4	Correctness proof	9
4.5	Complexity	9
4.6	C++ and Python notes	9
4.7	Wrong answers and targeted tests	9
5	C. Linear Diophantine	10
5.1	Restatement and traps	10
5.2	Derivation and pseudocode	10
5.3	Worked traces	10
5.4	Correctness proof	11
5.5	Complexity	11
5.6	C++ and Python notes	11
5.7	Wrong answers and targeted tests	11
6	D. Clock Offset	12
6.1	Restatement	12
6.2	Constraints and traps	12
6.3	Observations	12
6.4	Sample walk-through	13
6.5	Algorithm	14
6.6	Correctness proof	14
6.7	Complexity	15
6.8	C++ implementation notes	15
6.9	Python implementation notes	15
6.10	Full C++ solution	15
6.11	Full Python solution	16

6.12	Common wrong answers	17
6.13	Tests that matter	17
6.14	D-specific stress testing	18
7	E. Merge Congruences	19
7.1	Restatement and compatibility	19
7.2	Construction	19
7.3	Correctness proof	19
7.4	Complexity and tests	19
7.5	Full C++ solution	20
7.6	Full Python solution	20
8	F. Shared Maintenance Window	22
8.1	Finding the hidden congruences	22
8.2	Correctness proof	22
8.3	Complexity and tests	22
8.4	Full C++ solution	22
8.5	Full Python solution	23
9	Formative upsolve exercises	25
10	Local workflow	26
11	Carry-forward checklist	27
12	Full reference implementations	28
12.1	A. GCD LCM Queries: C++	28
12.2	A. GCD LCM Queries: Python	28
12.3	B. General Inverse: C++	29
12.4	B. General Inverse: Python	30
12.5	C. Linear Diophantine: C++	30
12.6	C. Linear Diophantine: Python	31
12.7	D. Clock Offset: C++	32
12.8	D. Clock Offset: Python	33
12.9	E. Merge Congruences: C++	33
12.10	E. Merge Congruences: Python	34
12.11	F. Shared Maintenance Window: C++	35
12.12	F. Shared Maintenance Window: Python	36

1 Editorial map

Problem	Constraints that choose the method	Core result
A. GCD LCM Queries	$q \leq 2 \cdot 10^5$, positive a, b $\leq 10^9$	Euclid, then $(a/g) \cdot b$
B. General Inverse	$q \leq 2 \cdot 10^5$, arbitrary positive $m \leq 10^9$	extended Euclid, not Fermat
C. Linear Diophantine	$q \leq 2 \cdot 10^5$, positive a, b, c $\leq 10^9$	gcd divisibility, then Bezout scaling
D. Clock Offset	$q \leq 2 \cdot 10^5$, arbitrary positive $a, b, m \leq 10^9$	reduce a linear congruence by its gcd
E. Merge Congruences	two arbitrary moduli up to 10^9	generalized CRT derived from Diophantine solving
F. Shared Maintenance Window	periodic events plus $\text{earliest} \leq 10^{18}$	recognize CRT, then advance by its lcm period

All six problems are designed for $O(\log \text{ value})$ work per query. A linear search for a divisor, inverse, or coefficient pair is not competitive at these limits.

2 Shared tool: extended Euclid

Use this contract throughout the section:

```
extended_gcd(a,b) returns (g,x,y) such that  
g = gcd(a,b) and  $a*x + b*y = g$ .
```

The recursive transition is:

```
if b == 0: return (a,1,0)  
(g,x1,y1) = extended_gcd(b,a mod b)  
return (g,y1,x1-floor(a/b)*y1)
```

The last line is not a memorized trick. It follows from replacing $a \bmod b$ by $a - \text{floor}(a/b)*b$ in the recursive Bezout identity. Re-derive it whenever you are unsure which coefficient belongs to a .

2.1 Shared C++ helper

```
long long egcd(long long a, long long b, long long &x, long long &y) {  
    if (b == 0) {  
        x = 1;  
        y = 0;  
        return a;  
    }  
    long long x1, y1;  
    long long g = egcd(b, a % b, x1, y1);  
    x = y1;  
    y = x1 - (a / b) * y1;  
    return g;  
}
```

2.2 Shared Python helper

```
def egcd(a: int, b: int) -> tuple[int, int, int]:  
    if b == 0:  
        return a, 1, 0  
    g, x1, y1 = egcd(b, a % b)  
    return g, y1, x1 - (a // b) * y1
```

For this section's positive inputs both versions return a positive gcd. In a negative-input variant, define and enforce a positive-gcd contract before reusing this code.

3 A. GCD LCM Queries

3.1 Restatement and traps

For every positive pair (a,b) , print:

```
gcd(a,b) lcm(a,b)
```

The trap is intermediate overflow. Although the local maximum lcm is at most 10^{18} and fits in signed `long long`, $a*b$ must not be formed before removing the gcd. Also do not try to enumerate divisors up to $\sqrt{a}$ for every query: $2*10^5$ queries make that approach too slow.

3.2 Derivation and pseudocode

Euclid preserves common divisors under $(a,b) \rightarrow (b, a \bmod b)$. Once the second entry is zero, the first is the gcd. The prime-exponent identity gives:

```
lcm(a,b) = a/gcd(a,b) * b
```

Pseudocode:

```
for each (a,b):  
    g = gcd(a,b)  
    l = (a/g)*b  
    print g, l
```

3.3 Worked trace

For $a=84$, $b=126$:

```
gcd(84,126) = gcd(126,84) = gcd(84,42) = gcd(42,0) = 42  
lcm = (84/42)*126 = 252
```

The result passes two useful checks: both inputs divide 252, and $42*252 = 84*126$.

3.4 Correctness proof

Euclid's recurrence preserves the set of common divisors because $a \bmod b = a - qb$; hence it returns $\gcd(a, b)$. For each prime, $\gcd$ takes the smaller input exponent and lcm takes the larger one. Their exponent sum equals the exponent in $a * b$, proving $\gcd(a, b) * \text{lcm}(a, b) = a * b$. Therefore $(a/g) * b$ equals the lcm, and the algorithm prints both required values. $\square$

3.5 Complexity

Each $\gcd$ takes $O(\log \min(a, b))$; lcm arithmetic is constant time. Total time is $O(q \log \max(a, b))$ and extra memory is $O(1)$.

3.6 C++ and Python notes

In C++, `std::gcd(a, b)` and `long long l = a / g * b;` are enough under the local bounds. Python uses `math.gcd` and has arbitrary precision, but the same operation order documents the intended safe arithmetic.

```
long long g = gcd(a, b);  
long long l = a / g * b;
```

```
g = math.gcd(a, b)  
l = a // g * b
```

3.7 Wrong answers and targeted tests

Wrong approach	Input	Why it fails
$a * b / g$ in narrow arithmetic	large coprime values	multiplication can overflow first
output $a * b$ as lcm	12 18	common factors are counted twice
trial-divide every query	many near- 10^9 queries	query total is too large

Test equal numbers ($10^9, 10^9$), coprime numbers (7, 5), and one number dividing the other (9, 45).

4 B. General Inverse

4.1 Restatement and traps

For each positive pair (a,m) , print the smallest nonnegative x satisfying:

$$a \cdot x \equiv 1 \pmod{m}$$

or -1 when no inverse exists. The modulus is not guaranteed prime, so Fermat's a^{m-2} formula is invalid in general. The local contract explicitly uses 0 when $m=1$.

4.2 Derivation and pseudocode

An inverse exists precisely when there are integers x,y such that:

$$a \cdot x + m \cdot y = 1$$

Extended Euclid creates that equation exactly when $\gcd(a,m)=1$.

```
for each (a,m):
    if m == 1:
        print 0
        continue
    (g,x,y) = extended_gcd(a,m)
    if g != 1:
        print -1
    else:
        print normalized residue of x modulo m
```

For C++, normalize with $(x \% m + m) \% m$ or the two-step version from the lesson. Python's $x \% m$ is already nonnegative for positive m .

4.3 Worked traces

4.3.1 Inverse exists: $a=10, m=17$

Back-substitution gives:

$$1 = 2 \cdot 17 - 3 \cdot 10$$

Thus $x=-3$. Normalize: $-3 \bmod 17 = 14$. Check $10 \cdot 14 = 140 \equiv 1 \pmod{17}$.

4.3.2 Inverse does not exist: $a=6, m=9$

$\text{gcd}(6,9)=3$, so every value $6x+9y$ is divisible by three. It cannot equal one; print -1 without searching residues.

4.4 Correctness proof

If the algorithm prints a normalized coefficient x , extended Euclid gave $a*x+m*y=1$; reducing modulo m proves $a*x \equiv 1 \pmod{m}$. Conversely, if an inverse exists, rearranging its congruence gives an equation $a*x+m*y=1$. Bezout's identity then requires $\text{gcd}(a,m)=1$. Thus the gcd test rejects exactly the impossible cases and the returned coefficient is exactly the required inverse. $\square$

4.5 Complexity

Extended Euclid takes $O(\log \min(a,m))$ time and $O(\log \min(a,m))$ recursion depth, with $O(1)$ auxiliary data at each level. Iterative implementations can make the stack use $O(1)$.

4.6 C++ and Python notes

Do not call $\text{pow}(a,m-2,m)$ unless the statement establishes a prime modulus. In C++, retain the signed x from egcd until after the gcd check. In Python, the tuple helper directly returns (g,x,y) .

```
long long g = egcd(a, m, x, y);
if (g != 1) cout << -1;
else cout << (x % m + m) % m;
```

```
g, x, _ = egcd(a, m)
answer = -1 if g != 1 else x % m
```

4.7 Wrong answers and targeted tests

Wrong approach	Input	Why it fails
use Fermat for every modulus	6 35	35 is composite, though inverse exists
print raw coefficient	10 17	-3 is valid but not the requested representative
return an inverse when gcd is not one	6 9	no such residue exists
omit local modulus-one branch	25 1	required output is 0

Test a coprime pair with a negative Bezout coefficient, a shared-factor pair, $a=1$, and $m=1$.

5 C. Linear Diophantine

5.1 Restatement and traps

For each positive triple (a,b,c) , print any integers (x,y) satisfying:

$$a*x + b*y = c$$

or print IMPOSSIBLE. The variables are integers, not nonnegative integers; negative answers are often necessary. Do not divide c by the gcd until after checking that the division is exact.

5.2 Derivation and pseudocode

Let $(g,x_0,y_0) = \text{extended_gcd}(a,b)$. The equation is solvable iff $c \% g == 0$. When that condition holds, scale the Bezout equation by c/g .

```
for each (a,b,c):
    (g,x0,y0) = extended_gcd(a,b)
    if c mod g != 0:
        print IMPOSSIBLE
    else:
        scale = c/g
        print x0*scale, y0*scale
```

5.3 Worked traces

5.3.1 Solvable: $6x + 10y = 14$

Extended Euclid gives $6*2+10*(-1)=2$. Since $14/2=7$, scale the pair:

$$\begin{aligned} x &= 14, \quad y = -7 \\ 6*14 + 10*(-7) &= 14 \end{aligned}$$

5.3.2 Impossible: $6x + 10y = 15$

The left side is always divisible by two, while fifteen is not. The gcd test therefore rejects the equation before any coefficient is printed.

5.4 Correctness proof

Every integer combination $a*x+b*y$ is divisible by $g=\gcd(a,b)$, so if g does not divide c , no solution exists. If it does divide c , extended Euclid returns $a*x_0+b*y_0=g$; multiplying by c/g gives:

$$a*(x_0*c/g) + b*(y_0*c/g) = c.$$

Thus the algorithm prints IMPOSSIBLE exactly when necessary and otherwise prints a valid integer solution. $\square$

5.5 Complexity

One extended-gcd call costs $O(\log \min(a,b))$ per query. Scaling and printing are constant time; total extra memory is the recursion stack.

5.6 C++ and Python notes

Use `long long` for the local bounds. The local construction can produce negative values, so do not use unsigned types. In a higher-bound version, cast the scale multiplication to `__int128` in C++, and decide how the output contract handles values outside signed 64-bit range.

```
long long g = egcd(a, b, x, y);
if (c % g != 0) cout << "IMPOSSIBLE\n";
else cout << x * (c / g) << ' ' << y * (c / g) << '\n';
```

```
g, x, y = egcd(a, b)
if c % g:
    print("IMPOSSIBLE")
else:
    scale = c // g
    print(x * scale, y * scale)
```

5.7 Wrong answers and targeted tests

Wrong approach	Input	Why it fails
require nonnegative x,y	6 10 14	a valid construction has $y=-7$
scale without divisibility check	6 10 15	$15/2$ is not an integer scale
solve by bounded brute force	coefficients near 10^9	no useful finite search range
swap returned coefficients	15 21 12	substituted equation exposes the mistake

Always test the exact equation after printing. Include a case with $\gcd=1$, a case where $\gcd$ does not divide c , and a case whose valid output has a negative coordinate.

6 D. Clock Offset

6.1 Restatement

For every query, find the least non-negative x satisfying:

$$a \cdot x \equiv b \pmod{m}.$$

The output is -1 when no such residue exists. Unlike the inverse problem, the right-hand side is not necessarily one and a need not be coprime with m .

This is the blind exercise for the section. The statement gives only the congruence; it does not reveal the reduction needed to solve it.

6.2 Constraints and traps

With up to $2 \cdot 10^5$ queries and values up to 10^9 , trying candidates $x = 0, 1, 2, \dots$ can take a billion checks for one query. The intended work is a single gcd computation per query.

The case $\gcd(a, m) > 1$ is not automatically impossible. For example, $6x \equiv 8 \pmod{14}$ has the answer 6. The correct divisibility test involves the right-hand side too.

When a solution exists there are generally several integer solutions, but they are all equal modulo a smaller modulus. The problem asks for the representative in $[0, m/g - 1]$, where $g = \gcd(a, m)$, not merely any signed Bezout coefficient.

If $m/g = 1$, every integer is equivalent modulo one, and the requested least non-negative answer is 0. Both `% 1` in C++ and Python's `% 1` produce zero, so no separate special case is required.

6.3 Observations

The congruence means that m divides $a \cdot x - b$. Equivalently, there is some integer y such that:

$$a \cdot x + m \cdot y = b.$$

This is a linear Diophantine equation. Let $g = \gcd(a, m)$. It has a solution exactly when g divides b .

When it does, divide every term by g :

$$(a/g) \cdot x + (m/g) \cdot y = b/g.$$

Now a/g and m/g are coprime. Extended Euclid on the original pair gives coefficients p, q with $a \cdot p + m \cdot q = g$. Multiplying by b/g gives one solution:

$$x_0 = p * (b/g).$$

It remains to prove which other values of x work. Set:

$$\begin{aligned} A &= a/g \\ M &= m/g \\ B &= b/g \end{aligned}$$

The reduced congruence is:

$$A*x == B \pmod{M}, \text{ with } \gcd(A,M)=1.$$

Let x_0 be the particular solution constructed above, and let x be any other solution. Then both satisfy the congruence. Subtract them:

$$\begin{array}{rcl} A*x & == & B \pmod{M} \\ A*x_0 & == & B \pmod{M} \\ \hline A*(x-x_0) & == & 0 \pmod{M} \end{array}$$

Thus M divides $A*(x-x_0)$. Since A and M are coprime, Euclid's lemma forces M to divide $x-x_0$. Therefore every solution has the form:

$$x = x_0 + M*t = x_0 + (m/g)*t$$

for some integer t . Conversely, adding $M*t$ always preserves the original congruence because:

$$a*(M*t) = a*(m/g)*t = m*(a/g)*t,$$

which is a multiple of m . Hence this formula gives all solutions and no others.

Now divide x_0 by the positive period M :

$$x_0 = M*k + r, \text{ where } 0 \leq r < M.$$

The value $r = x_0 \bmod M$ belongs to the solution family. It is also the smallest non-negative member: subtracting one more M makes it negative, while adding M only makes it larger. Therefore the required answer is:

$$x_0 \bmod (m/g).$$

6.4 Sample walk-through

Consider $6x == 8 \pmod{14}$.

```
gcd(6,14) = 2, and 2 divides 8, so a solution may exist.  
6*5 + 14*(-2) = 2.
```

Scale this Bezout identity by $8/2 = 4$:

```
6*20 + 14*(-8) = 8.
```

Thus $x = 20$ works. The reduced modulus is $14/2 = 7$, and $20 \bmod 7 = 6$. Indeed, $6*6 = 36$, which is 8 modulo 14. The complete solution family is $x = 6 + 7t$, so 6 is the requested smallest non-negative value.

For $6x \equiv 7 \pmod{14}$, the gcd is still two, but two does not divide seven. The associated equation $6x + 14y = 7$ therefore has no integer solution.

6.5 Algorithm

```
for each query (a, b, m):  
    (g, p, q) = extended_gcd(a, m)  
    if b mod g != 0:  
        print -1  
        continue  
  
    reduced_modulus = m / g  
    x0 = p * (b / g)  
    answer = x0 mod reduced_modulus, normalized into [0, reduced_modulus)  
    print answer
```

For the positive inputs in this task, the standard extended-gcd helper returns the positive g . In C++, normalize a potentially negative remainder with:

```
answer %= reduced_modulus;  
if (answer < 0) answer += reduced_modulus;
```

Python's `%` already returns a non-negative remainder for a positive modulus.

6.6 Correctness proof

We prove that the algorithm prints the required answer for each query.

Lemma 1. The congruence $a*x \equiv b \pmod{m}$ has an integer solution if and only if $\gcd(a,m)$ divides b .

Proof. The congruence is equivalent to the existence of y with $a*x + m*y = b$. Every left-hand side is divisible by $\gcd(a,m)$, proving necessity. Conversely, extended Euclid produces p, q such that $a*p + m*q = g$, where $g = \gcd(a,m)$. If g divides b , multiplying this identity by b/g produces an equation with right-hand side b , proving sufficiency. $\square$

Lemma 2. If a solution exists, $x_0 = p \cdot (b/g)$ produced by the algorithm is a solution, and every solution has the same residue modulo m/g .

Proof. The scaled Bezout identity in Lemma 1 gives $a \cdot x_0 + m \cdot (q \cdot b/g) = b$, so x_0 satisfies the congruence. For any two solutions x_1, x_2 , subtract their congruences to get m divides $a \cdot (x_1 - x_2)$. Dividing a and m by their gcd leaves coprime integers, so m/g divides $x_1 - x_2$. Thus all solutions share one residue modulo m/g . $\square$

Theorem. For each query, the algorithm prints -1 exactly when no solution exists; otherwise it prints the smallest non-negative solution.

Proof. By Lemma 1, the algorithm prints -1 exactly in the impossible case. In the solvable case, Lemma 2 says x_0 is a solution and all solutions are congruent modulo m/g . Normalizing x_0 produces the unique representative of that residue in $[0, m/g)$, which is necessarily the smallest non-negative solution. $\square$

6.7 Complexity

Each query runs extended Euclid once:

```
Time:    O(log min(a, m)) per query
Memory: O(log min(a, m)) recursion stack, or O(1) with iterative Euclid
```

The aggregate time is $O(q \log 10^9)$, which is appropriate for the query limit.

6.8 C++ implementation notes

The scaled coefficient has magnitude below the signed 64-bit limit under the local bounds: both the Bezout coefficient and b/g are at most on the order of 10^9 . Keep all arithmetic in `long long`, never `int`.

Do not compute `a * answer` merely to verify the result in a 32-bit type. For a debug assertion with larger bounds, use `__int128` for that product.

6.9 Python implementation notes

Use an output list and one final `"\n".join(...)` for $2 \cdot 10^5$ answers. Python integers make the scale multiplication exact. Its positive-modulus remainder rule means `(coefficient * scale) % reduced_modulus` is already the required representative.

6.10 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

long long extended_gcd(long long a, long long b, long long& x, long long& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = extended_gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b, m;
        cin >> a >> b >> m;

        long long coefficient, unused;
        long long g = extended_gcd(a, m, coefficient, unused);
        if (b % g != 0) {
            cout << -1 << '\n';
            continue;
        }

        long long reduced_modulus = m / g;
        long long scale = b / g;
        long long answer = (coefficient * scale) % reduced_modulus;
        if (answer < 0) answer += reduced_modulus;
        cout << answer << '\n';
    }
    return 0;
}

```

6.11 Full Python solution

```

import sys

def extended_gcd(a: int, b: int) -> tuple[int, int, int]:

```

```

    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    values = iter(data[1:])
    answer: list[str] = []
    for _ in range(q):
        a = next(values)
        b = next(values)
        m = next(values)
        g, coefficient, _ = extended_gcd(a, m)
        if b % g:
            answer.append("-1")
            continue
        reduced_modulus = m // g
        answer.append(str((coefficient * (b // g)) % reduced_modulus))
    sys.stdout.write("\n".join(answer) + "\n")

if __name__ == "__main__":
    main()

```

6.12 Common wrong answers

Requiring $\gcd(a, m) = 1$, which rejects valid cases such as $6x \equiv 8 \pmod{14}$.
 Checking $\gcd(a, m)$ divides a rather than the right-hand side b .
 Using an inverse of a modulo m before dividing out the $\gcd$.
 Printing a negative Bezout-derived x instead of normalizing by m/g .
 Normalizing by m instead of m/g , which can return a valid but non-minimal x .

6.13 Tests that matter

$6x \equiv 8 \pmod{14}$: solvable even though $\gcd$ is 2.
 $6x \equiv 7 \pmod{14}$: impossible because 2 does not divide 7.
 Any solvable query with $m = \gcd(a, m)$: the answer must be 0.
 A coprime query with a negative Bezout coefficient: checks normalization.
 200,000 queries with $a = 1$: checks input/output throughput.

6.14 D-specific stress testing

For tiny moduli, an independent oracle needs no number theory: scan every candidate x from 0 through $m-1$ and keep the first for which $a*x \bmod m == b \bmod m$. If none works, the expected answer is -1 . This direct enumeration cross-checks both the divisibility rejection and the minimal-residue normalization without reusing extended Euclid.

The separate scale case uses $a = 1$. Then $x == b \pmod{m}$, so the known smallest answer is simply $b \bmod m$ for each of 200000 queries. It exercises input/output and the logarithmic extended-gcd loop while retaining a direct expected output.

7 E. Merge Congruences

7.1 Restatement and compatibility

Write a value satisfying the first congruence as $x=r_1+m_1*k$. Requiring the second congruence gives

$$m_1*k - m_2*t = r_2-r_1$$

for some integer t . Therefore a solution exists exactly when $\gcd(m_1,m_2)$ divides r_2-r_1 .

7.2 Construction

If extended Euclid returns

$$m_1*p + m_2*q = g,$$

then $k=p*(r_2-r_1)/g$ satisfies the needed equation. Construct $x_0=r_1+m_1*k$ and normalize it modulo

$$\text{period} = \text{lcm}(m_1,m_2) = (m_1/g)*m_2.$$

For safer arithmetic, the reference first normalizes k modulo m_2/g and uses a widened type for multiplication.

7.3 Correctness proof

The gcd divisibility condition is necessary and sufficient because the merge equation is linear Diophantine. When it holds, the scaled Bezout coefficients construct a value satisfying both original congruences. Any two common values differ by a multiple of both moduli, hence by their lcm; adding the lcm also preserves both remainders. Reducing the constructed value modulo that period therefore gives the unique common value in $[0,\text{period})$, which is the smallest nonnegative solution.

7.4 Complexity and tests

Each query takes $O(\log \min(m_1,m_2))$ time and $O(\log \min(m_1,m_2))$ recursion depth; extra memory is logarithmic. Test coprime moduli, compatible and incompatible noncoprime moduli, equal moduli, modulus one, a negative initial construction, and moduli near 10^9 .

7.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
using int128 = __int128_t;

tuple<int64, int64, int64> extended_gcd(int64 a, int64 b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = extended_gcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    while (q--) {
        int64 r1, m1, r2, m2;
        cin >> r1 >> m1 >> r2 >> m2;
        auto [g, coefficient, unused] = extended_gcd(m1, m2);
        int64 difference = r2 - r1;
        if (difference % g != 0) {
            cout << -1 << '\n';
            continue;
        }

        int64 reduced_period = m2 / g;
        int64 k = (int128)coefficient * (difference / g) % reduced_period;
        if (k < 0) k += reduced_period;

        int64 period = (m1 / g) * m2;
        int64 answer = (r1 + (int128)m1 * k) % period;
        cout << answer << '\n';
    }
}
```

7.6 Full Python solution

```
import sys

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1
```

```

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    index = 1
    answers = []
    for _ in range(q):
        r1, m1, r2, m2 = data[index:index+4]
        index += 4
        g, coefficient, _ = extended_gcd(m1, m2)
        difference = r2 - r1
        if difference % g:
            answers.append("-1")
            continue
        reduced_period = m2 // g
        k = coefficient * (difference // g) % reduced_period
        period = m1 // g * m2
        answers.append(str((r1 + m1 * k) % period))
    print("\n".join(answers))

if __name__ == "__main__":
    main()

```

8 F. Shared Maintenance Window

8.1 Finding the hidden congruences

Machine A signals at exactly the nonnegative times congruent to `offsetA` modulo `periodA`; machine B gives the second congruence. First merge those two classes exactly as in E. If they are incompatible, no lower bound can make a common signal appear.

If they merge to smallest value `first` with common period `period`, every common signal time is

```
first + period*z
```

for a nonnegative integer `z`. We need the smallest one at least `earliest`. If `first` is already large enough, keep it. Otherwise use ceiling division:

```
jumps = ceil((earliest-first)/period)
answer = first + jumps*period.
```

For nonnegative integers, compute the ceiling as $(\text{needed} + \text{period} - 1) / \text{period}$.

8.2 Correctness proof

The CRT merge proves that the generated arithmetic progression contains every and only common signal time. When advancing is necessary, `jumps` is the smallest integer whose added periods reach the lower bound. The reported time is therefore valid, while using one fewer jump would remain below `earliest`. Thus it is the earliest permitted common signal.

8.3 Complexity and tests

Extended Euclid dominates at $O(\log \min(\text{periodA}, \text{periodB}))$ time per query. Memory is logarithmic. Test a lower bound before, equal to, and after the first common time; a jump across several periods; incompatible cycles; period one; and `earliest` near 10^{18} .

8.4 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
using int128 = __int128_t;

tuple<int64, int64, int64> extended_gcd(int64 a, int64 b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = extended_gcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    while (q--) {
        int64 r1, m1, r2, m2, earliest;
        cin >> r1 >> m1 >> r2 >> m2 >> earliest;
        auto [g, coefficient, unused] = extended_gcd(m1, m2);
        int64 difference = r2 - r1;
        if (difference % g != 0) {
            cout << -1 << '\n';
            continue;
        }

        int64 reduced_period = m2 / g;
        int64 k = (int128)coefficient * (difference / g) % reduced_period;
        if (k < 0) k += reduced_period;
        int64 period = (m1 / g) * m2;
        int64 first = (r1 + (int128)m1 * k) % period;

        int128 answer = first;
        if (answer < earliest) {
            answer += ((earliest - answer + period - 1) / period) * period;
        }
        cout << (int64)answer << '\n';
    }
}

```

8.5 Full Python solution

```

import sys

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0

```

```

g, x1, y1 = extended_gcd(b, a % b)
return g, y1, x1 - (a // b) * y1

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    index = 1
    answers = []
    for _ in range(q):
        r1, m1, r2, m2, earliest = data[index:index+5]
        index += 5
        g, coefficient, _ = extended_gcd(m1, m2)
        difference = r2 - r1
        if difference % g:
            answers.append("-1")
            continue
        period = m1 // g * m2
        k = coefficient * (difference // g) % (m2 // g)
        first = (r1 + m1 * k) % period
        if first < earliest:
            first += (earliest - first + period - 1) // period * period
        answers.append(str(first))
    print("\n".join(answers))

if __name__ == "__main__":
    main()

```

9 Formative upsolve exercises

1. For A, explain why computing $(a/g)*b$ is safer than $a*b/g$, then evaluate the lcm of 1,000,000,000 and 1,000,000,000.

Hint: cancel the gcd before multiplying.

Answer: the gcd is 10^9 , so the lcm is 10^9 ; multiplying first is an unnecessary overflow risk in smaller integer types.

2. For B, use extended Euclid to find the inverse of 11 modulo 26.

Hint: $26 = 2*11 + 4$, then continue Euclid.

Answer: $11*19 = 209 = 1 \pmod{26}$, so the normalized inverse is 19.

3. For C, decide whether $35x + 15y = 12$ is solvable before constructing any coefficients.

Hint: compute only the gcd first.

Answer: it is **not** solvable because $\gcd(35,15)=5$ does not divide 12. This is the point of performing the divisibility test first.

10 Local workflow

Run the complete local suite with the reference target:

```
CP_TARGET=solution python3 sections/30_gcd_lcm_diophantine/check.py
```

Then run focused cases while developing:

```
A: (12,18), (7,5), (109,109)  
B: (3,11), (10,17), (6,9), (25,1)  
C: (6,10,14), (6,10,15), (7,5,1)  
D: (6,8,14), (6,7,14), (10,1,17)  
E: (2,6,5,9), (1,4,2,6), equal moduli  
F: the same compatible cycles with earliest before and after the first match
```

For C, a valid judge must check the equation itself rather than require one particular Bezout parameterization. Different valid pairs are expected.

11 Carry-forward checklist

1. Preserve a gcd or Bezout invariant; do not search candidates blindly.
2. Divide before multiplying for lcm, and still audit the final product range.
3. A modular inverse is a gcd-one statement, not automatically a Fermat statement.
4. For $ax+by=c$, gcd divisibility is both the impossibility test and the bridge to construction.
5. Treat negative coefficients as normal integer solutions, then verify by substitution.
6. For $a \cdot x \equiv b \pmod{m}$, divide by $\gcd(a,m)$ only after checking that it divides b .
7. Two congruences are compatible iff their residue difference is divisible by the moduli gcd.
8. After a CRT merge, normalize by the lcm; that lcm is also the complete solution period.

12 Full reference implementations

12.1 A. GCD LCM Queries: C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b;
        cin >> a >> b;
        long long g = gcd(a, b);
        cout << g << ' ' << a / g * b << '\n';
    }
    return 0;
}
```

12.2 A. GCD LCM Queries: Python

```
import math
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, b = data[idx], data[idx + 1]
        idx += 2
        g = math.gcd(a, b)
        out.append(f"{g} {a // g * b}")
    print("\n".join(out))
```

```
if __name__ == "__main__":
    main()
```

12.3 B. General Inverse: C++

```
#include <bits/stdc++.h>
using namespace std;

long long egcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, m;
        cin >> a >> m;
        if (m == 1) {
            cout << 0 << '\n';
            continue;
        }
        long long x, y;
        long long g = egcd(a, m, x, y);
        if (g != 1) {
            cout << -1 << '\n';
        } else {
            x %= m;
            if (x < 0) x += m;
            cout << x << '\n';
        }
    }
    return 0;
}
```

12.4 B. General Inverse: Python

```
import sys

def egcd(a: int, b: int) -> tuple[int, int, int]:
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, m = data[idx], data[idx + 1]
        idx += 2
        if m == 1:
            out.append("0")
            continue
        g, x, _ = egcd(a, m)
        out.append(str(x % m) if g == 1 else "-1")
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

12.5 C. Linear Diophantine: C++

```
#include <bits/stdc++.h>
using namespace std;

long long egcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
```

```

    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b, c;
        cin >> a >> b >> c;
        long long x, y;
        long long g = egcd(a, b, x, y);
        if (c % g != 0) {
            cout << "IMPOSSIBLE\n";
        } else {
            long long scale = c / g;
            cout << x * scale << ' ' << y * scale << '\n';
        }
    }
    return 0;
}

```

12.6 C. Linear Diophantine: Python

```

import sys

def egcd(a: int, b: int) -> tuple[int, int, int]:
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, b, c = data[idx], data[idx + 1], data[idx + 2]
        idx += 3
        g, x, y = egcd(a, b)
        if c % g != 0:
            out.append("IMPOSSIBLE")

```

```

        else:
            scale = c // g
            out.append(f"{x * scale} {y * scale}")
        print("\n".join(out))

if __name__ == "__main__":
    main()

```

12.7 D. Clock Offset: C++

```

#include <bits/stdc++.h>
using namespace std;

long long extended_gcd(long long a, long long b, long long& x, long long& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = extended_gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b, m;
        cin >> a >> b >> m;

        long long coefficient, unused;
        long long g = extended_gcd(a, m, coefficient, unused);
        if (b % g != 0) {
            cout << -1 << '\n';
            continue;
        }

        long long reduced_modulus = m / g;
        long long scale = b / g;
        long long answer = (coefficient * scale) % reduced_modulus;
        if (answer < 0) answer += reduced_modulus;
    }
}

```

```

        cout << answer << '\n';
    }
    return 0;
}

```

12.8 D. Clock Offset: Python

```

import sys

def extended_gcd(a: int, b: int) -> tuple[int, int, int]:
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    values = iter(data[1:])
    answer: list[str] = []
    for _ in range(q):
        a = next(values)
        b = next(values)
        m = next(values)
        g, coefficient, _ = extended_gcd(a, m)
        if b % g:
            answer.append("-1")
            continue
        reduced_modulus = m // g
        answer.append(str((coefficient * (b // g)) % reduced_modulus))
    sys.stdout.write("\n".join(answer) + "\n")

if __name__ == "__main__":
    main()

```

12.9 E. Merge Congruences: C++

```

#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
using int128 = __int128_t;

```

```

tuple<int64, int64, int64> extended_gcd(int64 a, int64 b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = extended_gcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    while (q--) {
        int64 r1, m1, r2, m2;
        cin >> r1 >> m1 >> r2 >> m2;
        auto [g, coefficient, unused] = extended_gcd(m1, m2);
        int64 difference = r2 - r1;
        if (difference % g != 0) {
            cout << -1 << '\n';
            continue;
        }

        int64 reduced_period = m2 / g;
        int64 k = (int128)coefficient * (difference / g) % reduced_period;
        if (k < 0) k += reduced_period;

        int64 period = (m1 / g) * m2;
        int64 answer = (r1 + (int128)m1 * k) % period;
        cout << answer << '\n';
    }
}

```

12.10 E. Merge Congruences: Python

```

import sys

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    index = 1

```

```

answers = []
for _ in range(q):
    r1, m1, r2, m2 = data[index:index+4]
    index += 4
    g, coefficient, _ = extended_gcd(m1, m2)
    difference = r2 - r1
    if difference % g:
        answers.append("-1")
        continue
    reduced_period = m2 // g
    k = coefficient * (difference // g) % reduced_period
    period = m1 // g * m2
    answers.append(str((r1 + m1 * k) % period))
print("\n".join(answers))

if __name__ == "__main__":
    main()

```

12.11 F. Shared Maintenance Window: C++

```

#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
using int128 = __int128_t;

tuple<int64, int64, int64> extended_gcd(int64 a, int64 b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = extended_gcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    while (q--) {
        int64 r1, m1, r2, m2, earliest;
        cin >> r1 >> m1 >> r2 >> m2 >> earliest;
        auto [g, coefficient, unused] = extended_gcd(m1, m2);
        int64 difference = r2 - r1;
        if (difference % g != 0) {
            cout << -1 << '\n';
            continue;
        }

        int64 reduced_period = m2 / g;

```

```

    int64 k = (int128)coefficient * (difference / g) % reduced_period;
    if (k < 0) k += reduced_period;
    int64 period = (m1 / g) * m2;
    int64 first = (r1 + (int128)m1 * k) % period;

    int128 answer = first;
    if (answer < earliest) {
        answer += ((earliest - answer + period - 1) / period) * period;
    }
    cout << (int64)answer << '\n';
}
}

```

12.12 F. Shared Maintenance Window: Python

```

import sys

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    index = 1
    answers = []
    for _ in range(q):
        r1, m1, r2, m2, earliest = data[index:index+5]
        index += 5
        g, coefficient, _ = extended_gcd(m1, m2)
        difference = r2 - r1
        if difference % g:
            answers.append("-1")
            continue
        period = m1 // g * m2
        k = coefficient * (difference // g) % (m2 // g)
        first = (r1 + m1 * k) % period
        if first < earliest:
            first += (earliest - first + period - 1) // period * period
        answers.append(str(first))
    print("\n".join(answers))

if __name__ == "__main__":
    main()

```